palimpsestr: User Manual

Complete Reference Guide v0.9.0

Enzo Cocca

2026-03-08

Contents

1 Introduction **2**
1.1 Installation . . . 2
1.2 Required and optional packages . . . 2

2 Data Preparation **2**
2.1 Required columns . . . 2
2.2 Optional columns . . . 3
2.3 Notes on Z coordinates . . . 3
2.4 Simulating test data . . . 3

3 Core Analysis **3**
3.1 Fitting the SEF model: `fit_sef()` . . . 3
3.1.1 Parameters explained . . . 4
3.1.2 Output: the `sef_fit` object . . . 4
3.2 Phase count selection: `compare_k()` . . . 5
3.3 Key diagnostics . . . 5

4 Intrusion Detection **7**

5 Model Validation **7**

6 Structured Exports **7**
6.1 Summary per stratigraphic unit . . . 7
6.2 Phase transition matrix . . . 8
6.3 Export all results to CSV . . . 8

7 Harris Matrix Integration **8**
7.1 Auto-generate from depth ordering . . . 8
7.2 Import from CSV . . . 8
7.3 Validate phases against stratigraphy . . . 8

8 Visualization **9**
8.1 ggplot2 plots (static) . . . 9
8.2 Interactive plots (plotly) . . . 15
8.3 Base R plots . . . 15
8.4 GIS export . . . 19

9 Interpretive Report **19**

10 Database Import **21**

11 Phase Reordering **22**

12 Bootstrap Confidence Intervals **22**

13 Cross-Validation **22**

14 Data-Driven Weight Optimisation **23**

15 Sparse SEI for Large Datasets **23**

16 Complete Workflow Example **23**

17 Function Reference **24**
17.1 Core . . . 24
17.2 Diagnostics . . . 24
17.3 Validation . . . 24
17.4 Export . . . 24
17.5 Harris Matrix . . . 25
17.6 Robustness & Sensitivity . . . 25
17.7 SEI / ESE . . . 25
17.8 Plots (ggplot2) . . . 25
17.9 Plots (base R) . . . 25
17.10 GIS . . . 26
17.11 Other . . . 26

1 Introduction

palimpsestr is an R package for the probabilistic decomposition of archaeological palimpsests using the Stratigraphic Entanglement Field (SEF) framework. It integrates spatial proximity, vertical distribution, chronological overlap, and cultural similarity to estimate latent depositional phases.

1.1 Installation

```r
# From GitHub
devtools::install_github("enzococca/palimpsestr")

# From local source
install.packages("palimpsestr_0.9.0.tar.gz", repos = NULL, type = "source")
```

1.2 Required and optional packages

Package	Status	Purpose
stats, utils, graphics, grDevices	Required	Core R
ggplot2	Suggested	Publication-quality plots (`gg_*` functions)
plotly	Suggested	Interactive visualization (`as_plotly()`)
sf	Suggested	GIS export (`as_sf_phase()`, `as_sf_links()`)
DBI, RSQLite, RPostgres	Suggested	Database import (`read_db()`)

2 Data Preparation

2.1 Required columns

Every dataset must contain:

Column	Type	Description
`x`	numeric	Easting coordinate (metres)
`y`	numeric	Northing coordinate (metres)
`z`	numeric	Depth or elevation (metres)
`date_min`	numeric	Start of chronological interval
`date_max`	numeric	End of chronological interval
`class`	character	Material class (ceramic, lithic, bone, metal, etc.)

2.2 Optional columns

Column	Type	Description
`id`	any	Unique identifier for each find
`context`	character	Stratigraphic unit label (e.g., "SU_1")
`taf_score`	numeric (0–1)	Taphonomic disturbance score
`true_phase`	integer	Known phase (for validation only)

2.3 Notes on Z coordinates

Each artifact needs its own z value. If you don't have point-level depth measurements, you can assign the mean depth of the stratigraphic unit (US) to all finds within it:

```
# Example: assign US mean depth to finds
data$z <- ave(data$z_us, data$context, FUN = mean)
```

2.4 Simulating test data

```
# Well-separated phases (low mixing)
easy <- archaeo_sim(n = 150, k = 3, seed = 1, mixing = 0.05)

# Moderate palimpsest
moderate <- archaeo_sim(n = 200, k = 3, seed = 1, mixing = 0.30)

# Compressed palimpsest (hard to resolve)
compressed <- archaeo_sim(n = 250, k = 4, seed = 1, mixing = 0.50)
```

Parameters:

- `n`: number of artifacts
- `k`: number of phases
- `seed`: random seed for reproducibility
- `mixing`: proportion of inter-phase mixing (0 = none, 1 = complete)

3 Core Analysis

3.1 Fitting the SEF model: `fit_sef()`

```
fit <- fit_sef(
  data     = moderate,
  coords   = c("x", "y", "z"),        # coordinate columns
  chrono   = c("date_min", "date_max"), # chronological range columns
  class    = "class",                   # material class column
```

```r
  tafonomy = "taf_score",             # optional: taphonomic score
  context  = "context",               # optional: stratigraphic unit
  harris   = NULL,                    # optional: Harris Matrix penalty
  k        = 3,                       # number of phases
  weights  = c(ws = 1, wz = 1, wt = 1, wc = 1), # domain weights
  seed     = 42,                      # random seed
  em_iter  = 100,                     # max EM iterations
  em_tol   = 1e-5,                    # convergence tolerance
  n_init   = 5                        # number of random initialisations
)
print(fit)
#> <sef_fit>
#> Observations: 200
#> Estimated phases: 3
#> Mean entropy: 0.000
#> Mean local SEI: 117.643
#> Mean energy: 4.358
#> LogLik: -1416.627
#> BIC: 3129.960
#> ICL: 3129.920
#> PDI: 1.000
#> Converged: yes
#> Initialisations: 5
```

3.1.1 Parameters explained

Parameter	Default	Description
`k`	3	Number of depositional phases to estimate
`weights`	`c(ws=1, wz=1, wt=1, wc=1)`	Relative weights for spatial (`ws`), vertical (`wz`), temporal (`wt`), and cultural (`wc`) evidence
`em_iter`	100	Maximum EM iterations
`em_tol`	1e-5	Convergence tolerance (log-likelihood change)
`n_init`	1	Number of random initialisations (higher = more robust, slower)
`tafonomy`	NULL	Column name for taphonomic disturbance (0–1)
`context`	NULL	Column name for stratigraphic unit labels
`harris`	NULL	$n \times n$ penalty matrix (from `harris_from_contexts()` or `read_harris()`)

3.1.2 Output: the `sef_fit` object

The returned object contains:

Field	Type	Description
`$phase`	integer vector	Dominant phase assignment per find

Field	Type	Description
`$phase_prob`	matrix	$n \times k$ matrix of phase membership probabilities
`$entropy`	numeric vector	Shannon entropy per find
`$energy`	numeric vector	ESE per find
`$local_sei`	numeric vector	Local SEI per find
`$converged`	logical	Whether EM converged
`$n_init`	integer	Number of initialisations used
`$em_loglik`	numeric vector	Log-likelihood trace across iterations
`$model_stats`	list	Summary statistics (PDI, BIC, loglik, etc.)

3.2 Phase count selection: `compare_k()`

When K is unknown, compare multiple values:

```
ck <- compare_k(moderate, k_values = 2:5, seed = 42,
                tafonomy = "taf_score", context = "context")
print(ck)
#>   k       pdi mean_entropy mean_local_sei mean_energy      loglik       bic
#> 1 2 0.9929850 4.862417e-03       117.6428    4.358157 -1661.83619 3519.7101
#> 2 3 0.9999074 1.017010e-04       117.6428    4.358157 -1416.62731 3129.9604
#> 3 4 0.9999932 9.441871e-06       117.6428    4.358157  -588.74176 1574.8573
#> 4 5 0.9989364 1.711724e-03       117.6428    4.358157   -28.34952  554.7409
#>         icl tot_withinss pseudo_bic
#> 1 3517.7652     749.9725   359.7135
#> 2 3129.9197     545.5200   343.7390
#> 3 1574.8536     493.7309   371.4741
#> 4  554.0562     447.6697   399.5720
```

The optimal K minimises BIC while maximising PDI.

3.3 Key diagnostics

```
# Palimpsest Dissolution Index (global separability, 0--1)
pdi(fit)
#> [1] 0.9999074

# Compact summary
sef_summary(fit)
#> $n
#> [1] 200
#>
#> $k
#> [1] 3
#>
#> $pdi
#> [1] 0.9999074
#>
#> $mean_entropy
#> [1] 0.000101701
#>
#> $mean_energy
#> [1] 4.358157
#>
```

```
#> $loglik
#> [1] -1416.627
#>
#> $bic
#> [1] 3129.96
#>
#> $phase_count
#>
#>  1  2  3
#> 66 68 66

# Phase probability table
head(as_phase_table(fit))
#>     id dominant_phase phase1       phase2       phase3      entropy local_sei
#> 1 F001              1      1 3.464056e-17 9.447792e-41 1.312929e-15  126.0263
#> 2 F002              1      1 6.500699e-17 4.337547e-41 2.422943e-15  118.9763
#> 3 F003              1      1 2.885892e-19 1.470422e-45 1.231967e-17  103.0079
#> 4 F004              1      1 2.313963e-21 1.196827e-47 1.099487e-19  125.0059
#> 5 F005              1      1 4.942410e-14 1.953203e-35 1.563788e-12  130.4531
#> 6 F006              1      1 2.007763e-16 1.345985e-42 7.478970e-15  134.3008
#>     energy
#> 1 4.368913
#> 2 4.297416
#> 3 5.225635
#> 4 6.070146
#> 5 8.384327
#> 6 6.517593

# Full diagnostic table (input + diagnostics)
head(phase_diagnostic_table(fit))
#>     id        x         y        z context date_min date_max   class taf_score
#> 1 F001 12.36888 104.53174 11.79768    SU_4 970.6746 1011.055  lithic 0.4687699
#> 2 F002 19.12233  89.86743 19.78304    SU_3 960.5955 1080.232  lithic 0.3796706
#> 3 F003 24.19310  96.13386 18.97572    SU_7 966.4167 1030.552    bone 0.2288079
#> 4 F004 26.50473  99.62853 18.50500    SU_2 947.1761 1027.755  lithic 0.2246840
#> 5 F005 45.78809  91.97095 17.58970    SU_1 975.1793 1071.124 ceramic 0.1925518
#> 6 F006 32.65961  89.87875 17.67439    SU_4 946.7372 1055.110 ceramic 0.5558153
#>   true_phase dominant_phase phase1       phase2       phase3      entropy
#> 1          1              1      1 3.464056e-17 9.447792e-41 1.312929e-15
#> 2          1              1      1 6.500699e-17 4.337547e-41 2.422943e-15
#> 3          1              1      1 2.885892e-19 1.470422e-45 1.231967e-17
#> 4          1              1      1 2.313963e-21 1.196827e-47 1.099487e-19
#> 5          1              1      1 4.942410e-14 1.953203e-35 1.563788e-12
#> 6          1              1      1 2.007763e-16 1.345985e-42 7.478970e-15
#>   local_sei   energy
#> 1  126.0263 4.368913
#> 2  118.9763 4.297416
#> 3  103.0079 5.225635
#> 4  125.0059 6.070146
#> 5  130.4531 8.384327
#> 6  134.3008 6.517593
```

4 Intrusion Detection

```
di <- detect_intrusions(fit)
suspects <- di[di$intrusion_prob > 0.5, ]
cat("Suspected intrusions:", nrow(suspects), "\n")
#> Suspected intrusions: 6
head(suspects[order(suspects$intrusion_prob, decreasing = TRUE), ])
#>       id intrusion_prob
#> 28  F028      0.6138397
#> 159 F159      0.5937866
#> 168 F168      0.5589498
#> 22  F022      0.5433433
#> 142 F142      0.5359973
#> 36  F036      0.5063458
```

5 Model Validation

When true phase labels are known:

```
# Adjusted Rand Index (-1 to 1, 1 = perfect)
adjusted_rand_index(fit, moderate$true_phase)
#> [1] 0.9848866

# Confusion matrix (auto-reordered for best diagonal match)
confusion_matrix(fit, moderate$true_phase)
#>           true
#> estimated  1  2  3
#>         1 66  0  0
#>         2  1 67  0
#>         3  0  0 66
```

6 Structured Exports

6.1 Summary per stratigraphic unit

```
us_summary_table(fit)
#>   context n_finds dominant_phase    purity mean_entropy mean_energy
#> 1    SU_1      18              1 0.3888889 1.128512e-03    4.627645
#> 2    SU_2      29              3 0.3793103 2.769511e-13    4.122283
#> 3    SU_3      20              1 0.3500000 5.692683e-11    5.030029
#> 4    SU_4      20              2 0.4500000 5.459267e-09    4.676633
#> 5    SU_5      16              2 0.3750000 6.878401e-13    4.889321
#> 6    SU_6      29              2 0.3793103 1.140044e-10    4.346820
#> 7    SU_7      23              3 0.4347826 6.975216e-10    4.308097
#> 8    SU_8      16              2 0.5625000 5.610695e-07    3.935491
#> 9    SU_9      29              1 0.4137931 6.163456e-07    3.734943
#>   mean_local_sei n_intrusions
#> 1       120.8741            1
#> 2       114.3377            0
#> 3       113.3714            2
#> 4       114.9627            2
#> 5       117.8229            1
```

```
#> 6       118.1819            0
#> 7       121.6252            0
#> 8       125.1586            0
#> 9       115.7931            0
```

Columns: `context`, `n_finds`, `dominant_phase`, `purity`, `mean_entropy`, `mean_energy`, `mean_local_sei`, `n_intrusions`.

6.2 Phase transition matrix

Shows how often finds from phase i sit directly above finds from phase j:

```
phase_transition_matrix(fit)
#>        phase1 phase2 phase3
#> phase1     59      7      0
#> phase2      4     28     35
#> phase3      2     33     31
```

6.3 Export all results to CSV

```
export_results(fit, dir = "results/", format = "csv", prefix = "site_X")
```

Creates 4 files:

- `site_X_phases.csv` — phase assignments and diagnostics per find
- `site_X_intrusions.csv` — intrusion probabilities
- `site_X_us_summary.csv` — per-US aggregated statistics
- `site_X_model_summary.csv` — model-level metrics (n, k, PDI, BIC, etc.)

7 Harris Matrix Integration

7.1 Auto-generate from depth ordering

```
H <- harris_from_contexts(moderate, z_col = "z", context_col = "context",
                          penalty_scale = 0.5)
dim(H)
#> [1] 200 200
```

7.2 Import from CSV

The CSV must have columns `from`, `to`, and optionally `weight`:

```
from,to,weight
SU_1,SU_2,1
SU_2,SU_3,1
SU_1,SU_3,0.5
```

```
H <- read_harris("harris_matrix.csv", contexts = data$context)
fit <- fit_sef(data, k = 3, harris = H, context = "context")
```

7.3 Validate phases against stratigraphy

```
validate_phases_harris(fit)
#>   upper lower upper_phase lower_phase consistent
```

```
#> 1  SU_1  SU_2          1          3       TRUE
#> 2  SU_2  SU_5          3          2      FALSE
#> 3  SU_5  SU_7          2          3       TRUE
#> 4  SU_7  SU_6          3          2      FALSE
#> 5  SU_6  SU_4          2          2       TRUE
#> 6  SU_4  SU_9          2          1      FALSE
#> 7  SU_9  SU_3          1          1       TRUE
#> 8  SU_3  SU_8          1          2       TRUE
```

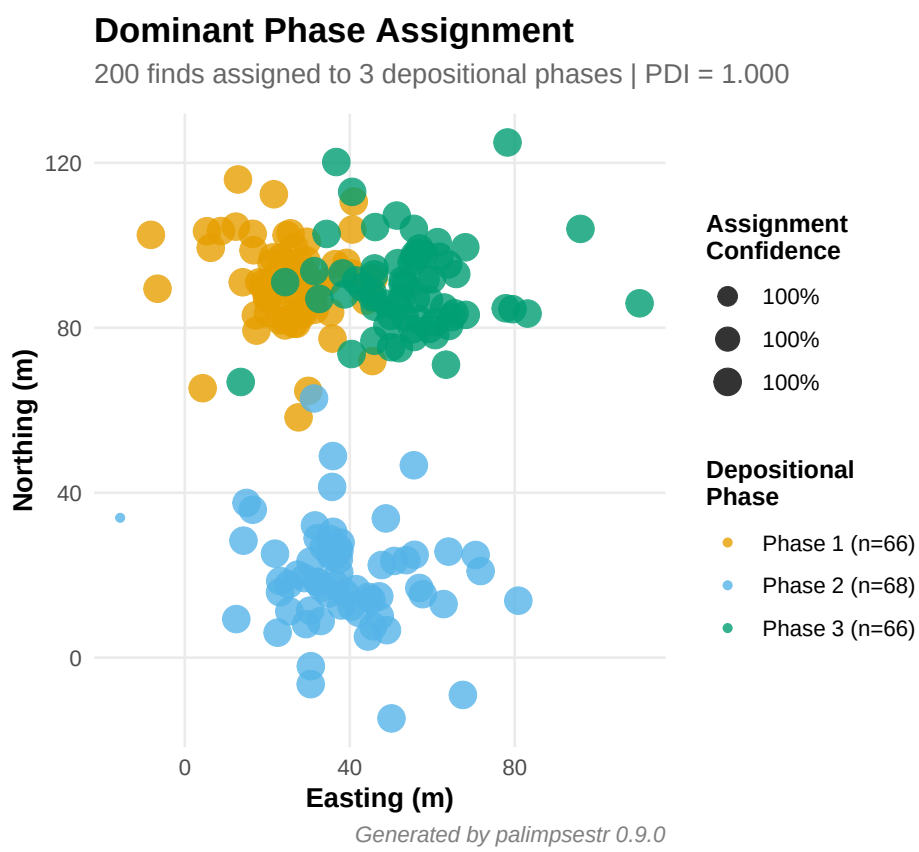
The `consistent` column flags whether the dominant phase ordering respects stratigraphic depth. `FALSE` indicates an inversion that may warrant investigation.

8 Visualization

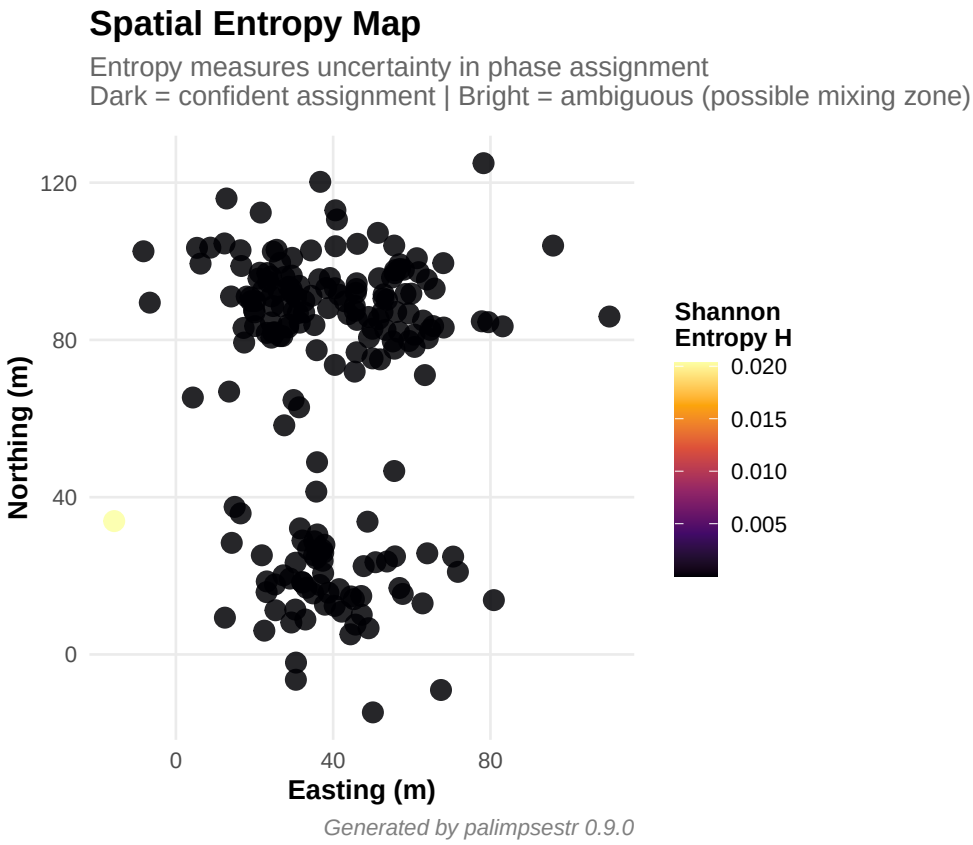
8.1 ggplot2 plots (static)

All `gg_*` functions require **ggplot2** and return ggplot objects.

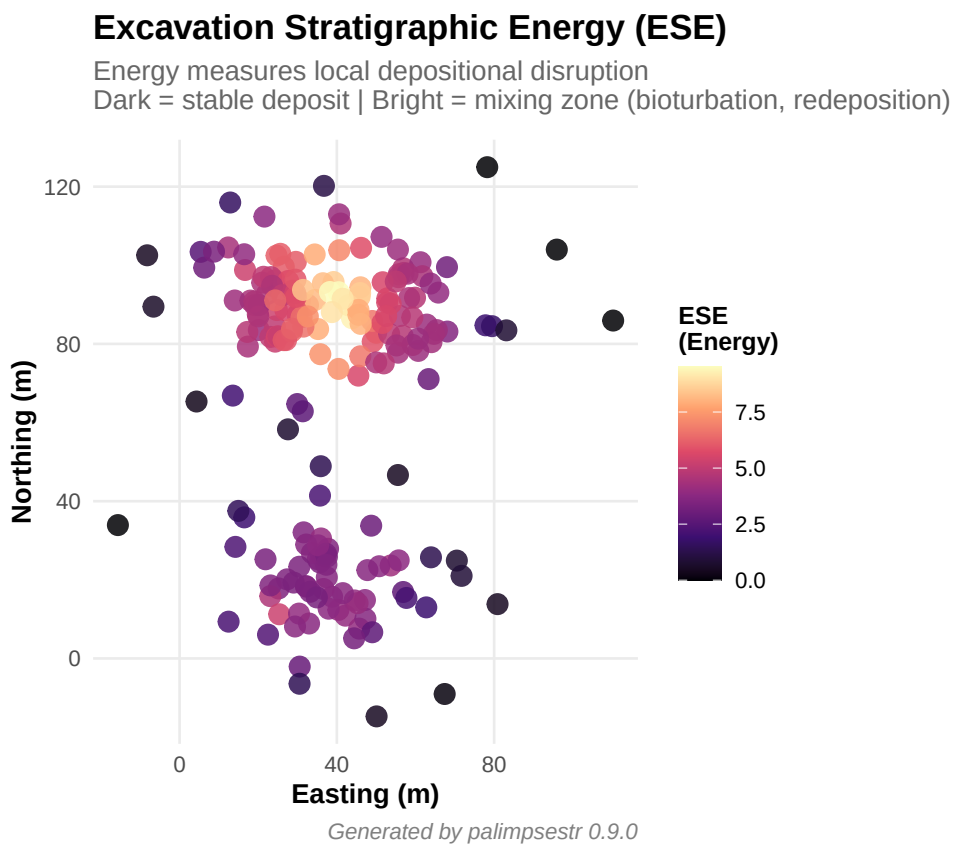
```
library(ggplot2)

# Phase assignment map
gg_phasefield(fit)
```



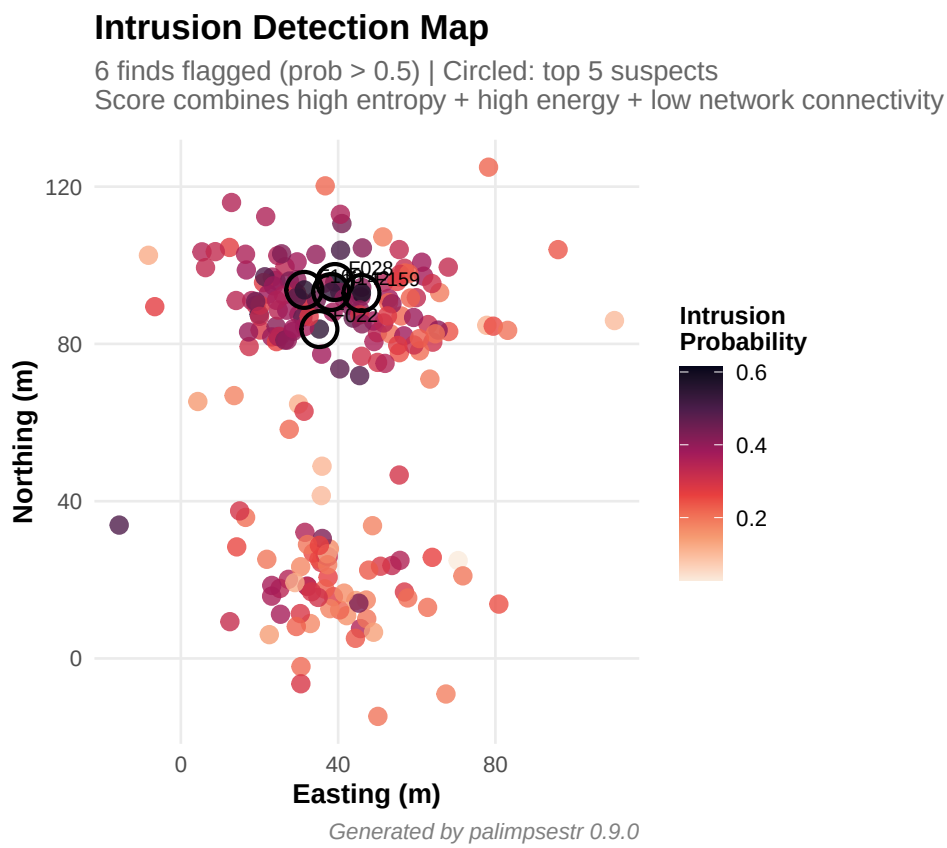

```
# Entropy map (uncertainty)
gg_entropy(fit)
```



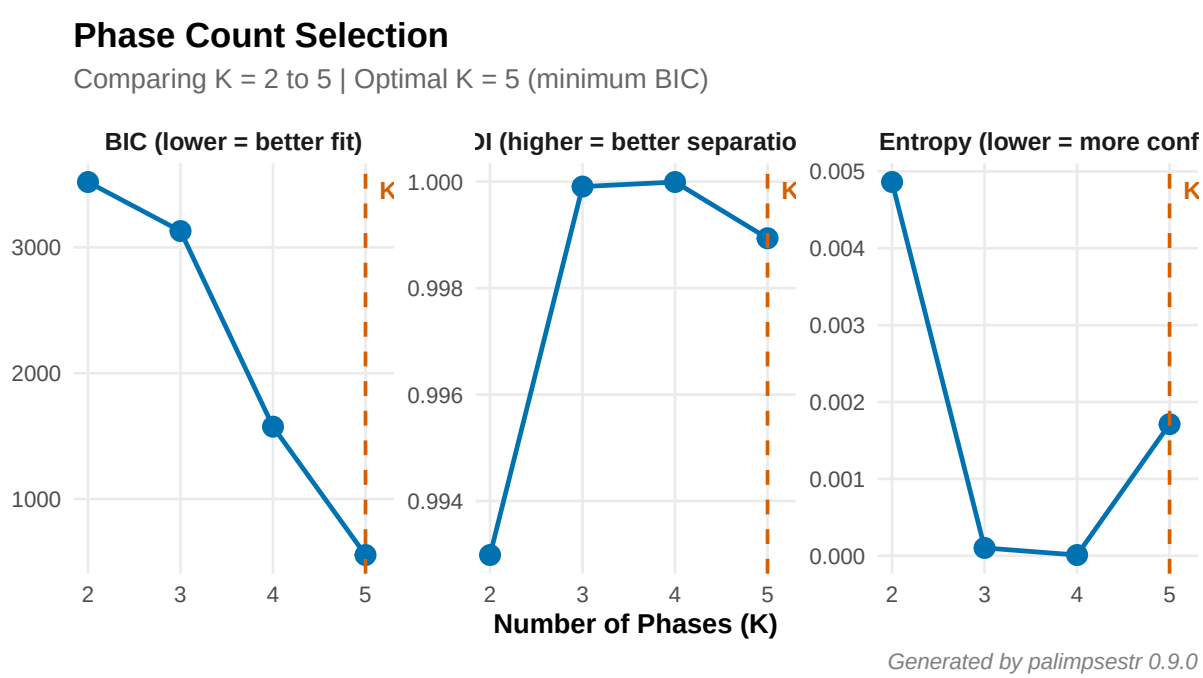

```
# Energy map (local disruption)
gg_energy(fit)
```



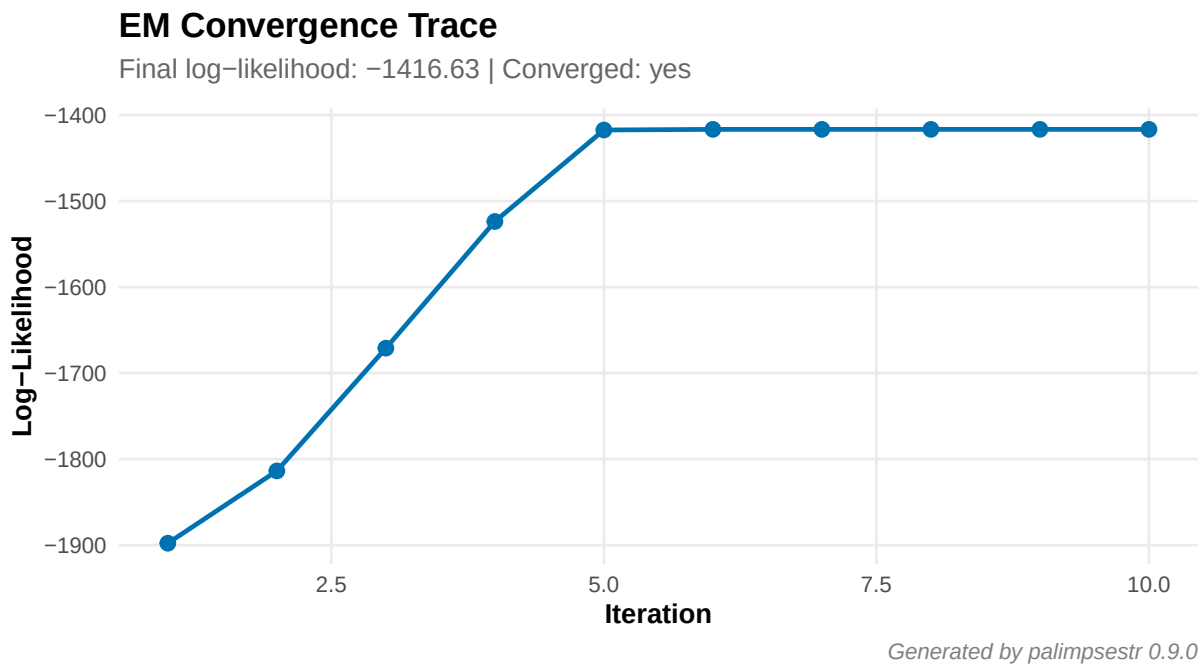

```
# Intrusion detection map
gg_intrusions(fit, top_n = 5)
```



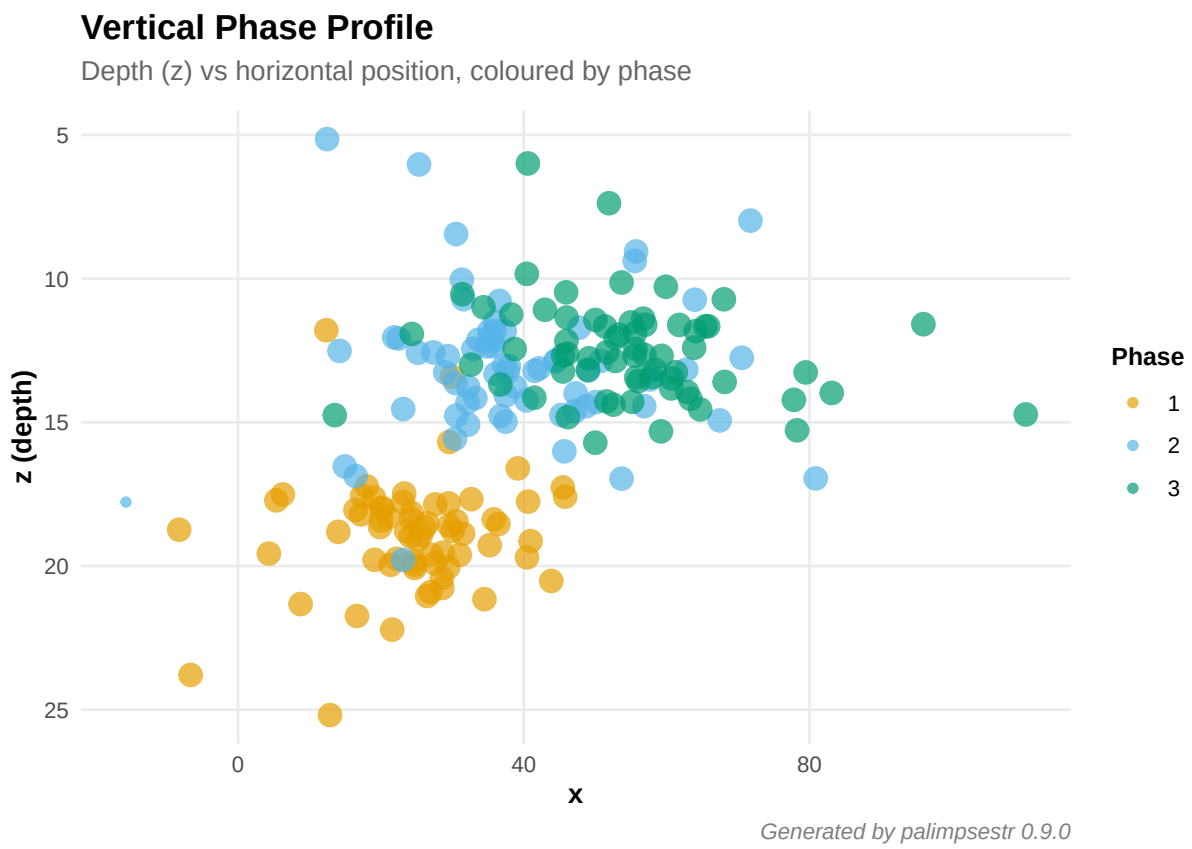

```
# Phase count selection diagnostics
gg_compare_k(ck)
```



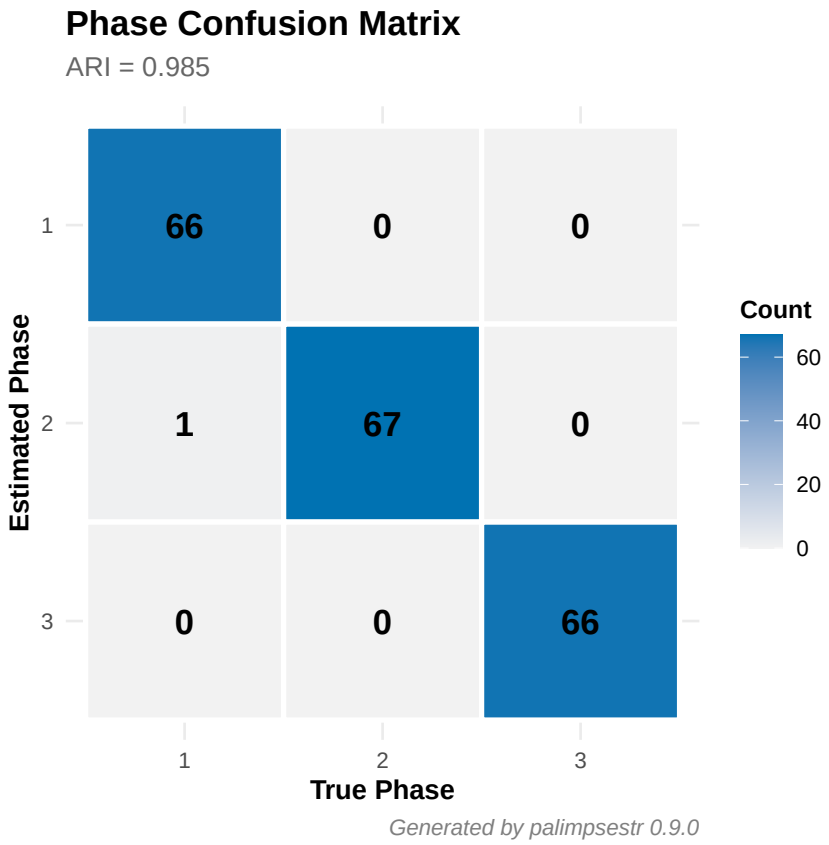
```r
# EM convergence trace
gg_convergence(fit)
```




```r
# Vertical phase profile (depth vs horizontal)
gg_phase_profile(fit)
```




```r
# Confusion matrix heatmap
gg_confusion(fit, moderate$true_phase)
```



8.2 Interactive plots (plotly)

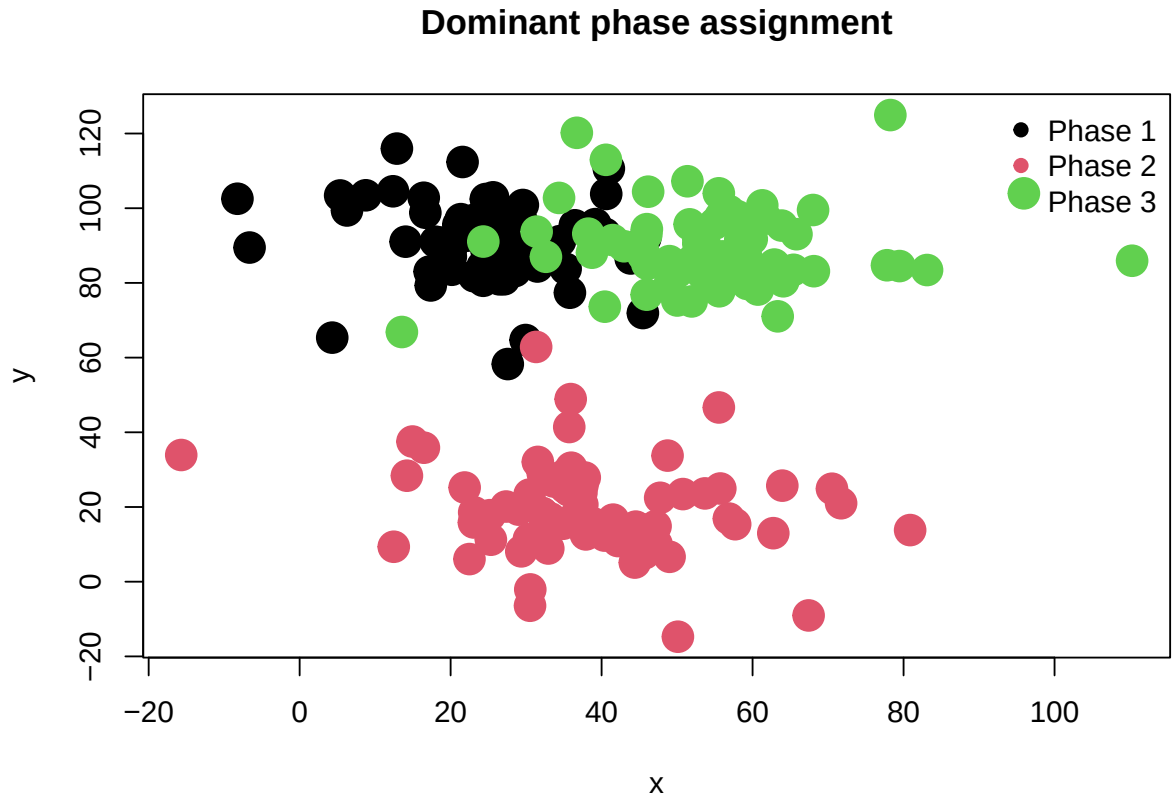
```
library(plotly)

# Convert any gg_* plot to interactive
as_plotly(gg_phasefield(fit))
as_plotly(gg_entropy(fit))
as_plotly(gg_intrusions(fit))
```

Hover tooltips show: ID, context, phase probability, dating, class, entropy, energy, intrusion score.

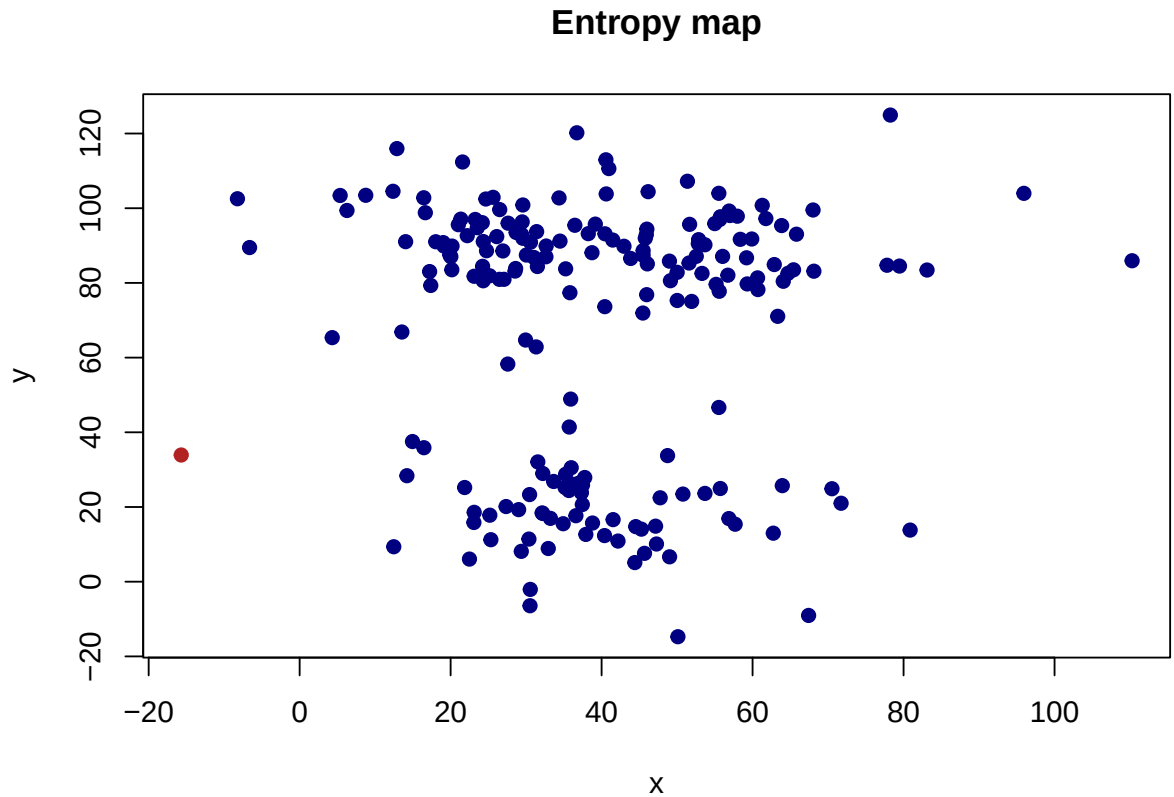
8.3 Base R plots

For environments without ggplot2:

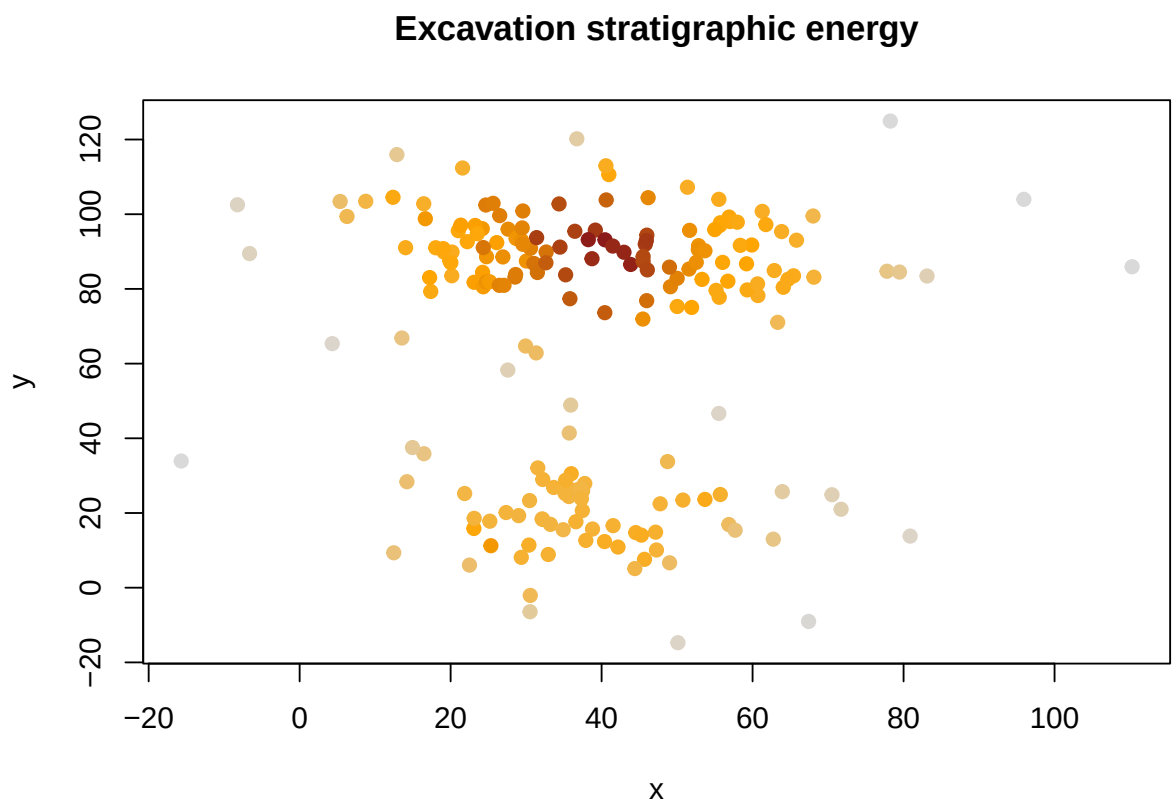
```
plot_phasefield(fit)
```




```r
plot_entropy(fit)
```




```
plot_energy(fit)
```




```
plot_sei_profile(fit)
```

8.4 GIS export

```
library(sf)

# Point layer with phase assignments
sf_pts <- as_sf_phase(fit, crs = 32632L, dims = "XYZ")
st_write(sf_pts, "results.gpkg", layer = "phases")

# High-SEI links as lines
sf_links <- as_sf_links(fit, quantile_threshold = 0.9, crs = 32632L)
st_write(sf_links, "results.gpkg", layer = "sei_links")
```

9 Interpretive Report

Generate a plain-language report for inclusion in excavation documentation:

```
report_sef(fit, lang = "en")
#> # SEF Analysis Report
#>
#> ## Overview
#>
#> The Stratigraphic Entanglement Field model with **K = 3 phases** was fitted to a dataset of **200 fi
#>
#> The Palimpsest Dissolution Index (PDI) is **1.000**, indicating **excellent phase separation**. Mean
```

```
#>
#> Model diagnostics: LogLik = -1416.6, BIC = 3130.0.
#>
#> ## Phase Composition
#>
#> ### Phase 1 (66 finds, 33.0%)
#>
#> - **Dating**: average 1001 CE (range: 894 to 1122)
#> - **Material classes**: bone (13), ceramic (20), lithic (24), metal (9)
#> - **Mean entropy**: 0.0000; **Mean energy**: 5.151
#> - **Contexts**: SU_1, SU_2, SU_3, SU_4, SU_5, SU_6, ... (9 total)
#> - **Assessment**: Entropy is very low, suggesting this phase is internally coherent and well-defined
#>
#> ### Phase 2 (68 finds, 34.0%)
#>
#> - **Dating**: average 1146 CE (range: 1004 to 1246)
#> - **Material classes**: bone (17), ceramic (25), lithic (14), metal (12)
#> - **Mean entropy**: 0.0003; **Mean energy**: 2.994
#> - **Contexts**: SU_1, SU_2, SU_3, SU_4, SU_5, SU_6, ... (9 total)
#> - **Assessment**: Entropy is very low, suggesting this phase is internally coherent and well-defined
#>
#> ### Phase 3 (66 finds, 33.0%)
#>
#> - **Dating**: average 1303 CE (range: 1211 to 1393)
#> - **Material classes**: bone (10), ceramic (22), lithic (20), metal (14)
#> - **Mean entropy**: 0.0000; **Mean energy**: 4.970
#> - **Contexts**: SU_1, SU_2, SU_3, SU_4, SU_5, SU_6, ... (9 total)
#> - **Assessment**: Entropy is very low, suggesting this phase is internally coherent and well-defined
#>
#> ## Intrusion Detection
#>
#> **6 finds** (3.0%) have an intrusion probability above 0.5, flagging them as potentially displaced o
#>
#> Top suspects:
#>
#> | ID | Context | Intrusion Prob. |
#> |---|---|---|
#> | F028 | SU_3 | 0.614 |
#> | F159 | SU_1 | 0.594 |
#> | F168 | SU_3 | 0.559 |
#> | F022 | SU_4 | 0.543 |
#> | F142 | SU_5 | 0.536 |
#> | F036 | SU_4 | 0.506 |
#>
#> ## Stratigraphic Unit Purity
#>
#> Of **9 stratigraphic units**: **0 pure** (>90%), **0 mixed** (70-90%), **9 palimpsest** (<70%).
#>
#> Units classified as palimpsest require special attention, as material from multiple depositional epi
#>
#> ## Recommendations
#>
#> The deposit shows good phase separation. The model can be used with confidence for phase attribution
```

```
#>
#> ---
#> *Report generated by palimpsestr 0.9.0*
```

```
report_sef(fit, lang = "it")
#> # Rapporto Analisi SEF
#>
#> ## Panoramica
#>
#> Il modello Stratigraphic Entanglement Field con **K = 3 fasi** e' stato applicato a un dataset di **
#>
#> Il Palimpsest Dissolution Index (PDI) e' **1.000**, indicando una separazione **eccellente** delle f
#>
#> Diagnostiche: LogLik = -1416.6, BIC = 3130.0.
#>
#> ## Composizione delle Fasi
#>
#> ### Fase 1 (66 reperti, 33.0%)
#>
#> - **Datazione media**: 1001 d.C.
#> - **Classi materiali**: bone (13), ceramic (20), lithic (24), metal (9)
#> - **Entropia media**: 0.0000; **Energia media**: 5.151
#> - **US**: SU_1, SU_2, SU_3, SU_4, SU_5, SU_6, ... (9 totali)
#>
#> ### Fase 2 (68 reperti, 34.0%)
#>
#> - **Datazione media**: 1146 d.C.
#> - **Classi materiali**: bone (17), ceramic (25), lithic (14), metal (12)
#> - **Entropia media**: 0.0003; **Energia media**: 2.994
#> - **US**: SU_1, SU_2, SU_3, SU_4, SU_5, SU_6, ... (9 totali)
#>
#> ### Fase 3 (66 reperti, 33.0%)
#>
#> - **Datazione media**: 1303 d.C.
#> - **Classi materiali**: bone (10), ceramic (22), lithic (20), metal (14)
#> - **Entropia media**: 0.0000; **Energia media**: 4.970
#> - **US**: SU_1, SU_2, SU_3, SU_4, SU_5, SU_6, ... (9 totali)
#>
#> ## Intrusioni
#>
#> **6 reperti** (3.0%) presentano probabilita' di intrusione superiore a 0.5.
#>
#> ## Purezza delle US
#>
#> Su **9 US**: **0 pure** (>90%), **0 miste** (70-90%), **9 palinsesto** (<70%).
#>
#> ---
#> *Rapporto generato da palimpsestr 0.9.0*
```

10 Database Import

Import data from SQLite or PostgreSQL:

```r
# SQLite
data <- read_db(
  con = DBI::dbConnect(RSQLite::SQLite(), "excavation.sqlite"),
  table = "finds"
)

# PostgreSQL (PyArchInit)
data <- read_db(
  con = DBI::dbConnect(RPostgres::Postgres(), dbname = "pyarchinit", host = "localhost"),
  table = "finds_table"
)
```

11 Phase Reordering

Mixture models suffer from *label switching*: phase numbers are arbitrary across runs. Use `reorder_phases()` to ensure phase 1 = deepest (oldest) and phase K = shallowest (most recent):

```r
fit <- reorder_phases(fit)
table(fit$phase)
#>
#>  1  2  3
#> 66 68 66
```

12 Bootstrap Confidence Intervals

Assess uncertainty on key metrics via bootstrap resampling:

```r
bs <- bootstrap_sef(fit, n_boot = 50, verbose = FALSE)
print(bs)
#>      statistic      estimate         lower        upper           se
#> 1          pdi  9.999074e-01  9.815071e-01   1.00000000 6.423186e-03
#> 2 mean_entropy  1.017010e-04  1.618468e-10   0.02031658 7.056591e-03
#> 3  mean_energy  4.358157e+00  3.894108e+00   4.58012027 1.845486e-01
#> 4       loglik -1.416627e+03 -1.420076e+03 984.92508485 9.560619e+02
```

Returns a data.frame with columns `metric`, `mean`, `se`, `ci_lower`, `ci_upper` for PDI, mean entropy, mean energy, loglik, and optionally ARI (if `true_labels` is provided).

Parameters:

Parameter	Default	Description
`n_boot`	100	Number of bootstrap replicates
`conf`	0.95	Confidence level for intervals
`true_labels`	NULL	Known true phases for ARI confidence interval
`verbose`	TRUE	Print progress messages

13 Cross-Validation

Evaluate model fit via k-fold cross-validation:

```r
cv <- cv_sef(moderate, k_values = 3, n_folds = 3, seed = 42)
print(cv)
#>   k fold train_loglik test_loglik train_pdi
```

```
#> 1 3    1    -896.3563   -497.5387 0.9999998
#> 2 3    2    -260.8705   -239.6114 0.9581275
#> 3 3    3    -780.5878   -385.7559 0.9997784
```

Use with multiple K values to find the best:

```
cv <- cv_sef(data, k_values = 2:5, n_folds = 5, seed = 42)
print(cv)  # shows mean_test_loglik per k
```

14 Data-Driven Weight Optimisation

Instead of manually tuning SEI weights, use `optimize_weights()` to find the best configuration via grid search:

```
opt <- optimize_weights(data, k = 3, n_folds = 3, seed = 42)
print(opt$best_weights)

# Re-fit with optimal weights
fit <- fit_sef(data, k = 3, weights = opt$best_weights)
```

15 Sparse SEI for Large Datasets

For datasets with n > 1000, the full $n \times n$ SEI matrix may be too large. Use `sei_sparse()` which automatically applies a distance threshold:

```
# Automatic sparse mode
sei <- sei_sparse(data, max_dist = 5.0)

# Or use sei_matrix() directly with max_dist
sei <- sei_matrix(data, max_dist = 5.0)
```

Pairs beyond `max_dist` Euclidean distance receive SEI = 0, dramatically reducing memory and computation time.

16 Complete Workflow Example

```
library(palimpsestr)

# 1. Load data
data <- read.csv("site_data.csv")

# 2. Select optimal K
ck <- compare_k(data, k_values = 2:6, context = "context",
                tafonomy = "taf_score", n_init = 3)
gg_compare_k(ck)
best_k <- ck$k[which.min(ck$bic)]

# 3. Fit with optimal K and multiple inits
fit <- fit_sef(data, k = best_k, context = "context",
               tafonomy = "taf_score", n_init = 5, seed = 42)

# 4. Inspect results
print(fit)
```

```r
gg_phasefield(fit)
gg_entropy(fit)
gg_convergence(fit)
gg_phase_profile(fit)

# 5. Intrusion detection
gg_intrusions(fit, top_n = 10)

# 6. Harris validation
validate_phases_harris(fit)

# 7. Export
export_results(fit, dir = "results/")
report_sef(fit, lang = "en")

# 8. GIS
sf_pts <- as_sf_phase(fit, crs = 32632L)
sf::st_write(sf_pts, "results.gpkg", layer = "phases")
```

17 Function Reference

17.1 Core

Function	Description
`fit_sef()`	Fit the SEF model
`compare_k()`	Compare multiple K values
`archaeo_sim()`	Simulate archaeological data
`reorder_phases()`	Relabel phases by mean depth (deepest = phase 1)

17.2 Diagnostics

Function	Description
`pdi()`	Palimpsest Dissolution Index
`sef_summary()`	Compact model summary
`as_phase_table()`	Phase probability table
`predict_phase()`	Alias for `as_phase_table()`
`phase_diagnostic_table()`	Full diagnostic table
`detect_intrusions()`	Intrusion detection

17.3 Validation

Function	Description
`adjusted_rand_index()`	Adjusted Rand Index
`confusion_matrix()`	Confusion matrix (auto-reordered)

17.4 Export

Function	Description
`us_summary_table()`	Per-US summary statistics
`phase_transition_matrix()`	Vertical transition counts
`export_results()`	Export all results to CSV

17.5 Harris Matrix

Function	Description
`harris_from_contexts()`	Auto-generate penalty from depths
`read_harris()`	Import Harris Matrix from CSV
`validate_phases_harris()`	Check phase–stratigraphy consistency

17.6 Robustness & Sensitivity

Function	Description
`bootstrap_sef()`	Bootstrap confidence intervals on PDI, entropy, ARI
`cv_sef()`	K-fold cross-validation for model selection
`optimize_weights()`	Data-driven SEI weight optimisation via grid search

17.7 SEI / ESE

Function	Description
`sei_matrix()`	Stratigraphic Entanglement Index matrix
`sei_sparse()`	Sparse SEI for large datasets (n > 1000)
`local_sei()`	Row sums of SEI matrix
`ese()`	Excavation Stratigraphic Energy

17.8 Plots (ggplot2)

Function	Description
`gg_phasefield()`	Phase assignment map
`gg_entropy()`	Entropy map
`gg_energy()`	ESE energy map
`gg_intrusions()`	Intrusion detection map
`gg_compare_k()`	K selection diagnostics
`gg_convergence()`	EM convergence trace
`gg_phase_profile()`	Vertical depth profile
`gg_confusion()`	Confusion matrix heatmap
`gg_map()`	Overlay on excavation plan
`as_plotly()`	Convert any gg_* to interactive

17.9 Plots (base R)

Function	Description
`plot_phasefield()`	Phase map
`plot_entropy()`	Entropy map
`plot_energy()`	Energy map
`plot_sei_profile()`	SEI profile

17.10 GIS

Function	Description
`as_sf_phase()`	Export as sf point layer
`as_sf_links()`	Export SEI links as sf lines
`load_geometries()`	Load excavation plan geometries

17.11 Other

Function	Description
`report_sef()`	Interpretive report (en/it)
`read_db()`	Import from SQLite/PostgreSQL